ADITYA UNIVERSITY

## SCHOOL OF COMPUTING

## Department of Computer Science and Engineering

## DATA ANALYSIS ESSENTIALS

# CERTIFICATE

This is to certify that the Cornerstone project work entitled ***"LOAN PREDICTION DATA ANALYSIS"*** is submitted by

| | |
|---|---|
| RAMOJU SURYA VENKATA SAI PRAKASH | 25B11CS809 |
| SANTHA THANUJA | 25B11CS855 |
| SALADI VEERA SAI ANUSHA | 25B11CS839 |
| NIMISHA MAITY | 25B11CS677 |

in partial fulfilment of the requirements for the award of the B.Tech. degree in Computer Science and Engineering in the academic year 2026-2027.

Project Guide
K. Vydehi, MTech(Ph. D)
Assistant Professor
Department of CSE

Head of the Department
Dr. Phani Sridhar Addepalli, Ph.D.
Assistant Professor
Department of CSE

# DECLARATION

We hereby declare that the Cornerstone project entitled "**Loan Prediction Data Analysis**" is original and is submitted to **ADITYA UNIVERSITY**, Surampalem, in partial fulfillment of the requirements for the award of the **B.Tech. Degree** in **Computer Science and Engineering**. We further declare that this project work has not been submitted, either full or in part, for the award of any degree in this or any other institution. We also confirm that this work complies with the Institutional Academic Integrity and AI Usage Policy.

Place: Surampalem
Date: 03-04-2026

By


| | |
|---|---|
| RAMOJU SURYA VENKATA SAI PRAKASH | 25B11CS809 |
| SANTHA THANUJA | 25B11CS855 |
| SALADI VEERA SAI ANUSHA | 25B11CS839 |
| NIMISHA MAITY | 25B11CS677 |

# ACKNOWLEDGEMENT

It is with immense pleasure that we express our sincere gratitude to our Cornerstone Project Guide, **K. Vydehi, Assistant Professor,** who has guided us throughout the project. Her valuable support, encouragement, and guidance have greatly contributed to the successful completion of this work.

We would like to thank our Deputy Heads of the Department, **Dr. Chandra Sekhar Kolli, Associate Professor, Dr Hari Jyothula, Associate Professor and Dr Pennada Siva Satya Prasad, Assistant Professor,** Department of Computer Science and Engineering, for their support and valuable suggestions.

Our deepest thanks to our Head of the Department **Dr. Phani Sridhar Addepalli, Assistant Professor,** for his inspiration and for providing all the necessary facilities and resources required for this project.

We also extend our sincere thanks to **Dr. M.V. Rajesh, Associate Dean, School of Computing,** for their encouragement and support during our project.

We would like to express our sincere gratitude to **Dr. G. Suresh, Registrar, Dr. A Ramesh, Pro Vice-Chancellor (Engineering & Sciences), Dr. S. Rama Sree, Pro Vice-Chancellor (Academics), Dr. M.B. Srinivas, Vice-Chancellor, Dr. M. Sreenivasa Reddy, Deputy Pro Chancellor and Management, Aditya University** for providing excellent infrastructure and state-of-the-art laboratories.

Finally, we would like to thank the **faculty members, lab technicians, non-teaching staff, and our friends** who have directly or indirectly supported us in completing this Cornerstone project successfully.

# ABSTRACT

The Loan Prediction Data Analysis System is developed to analyze loan applicant data and identify the factors influencing loan approval in financial institutions. With the increasing number of loan applications, manual evaluation becomes time consuming and may lead to inconsistencies. This project aims to preprocess, analyze, and visualize loan data to support basic loan approval prediction using Python. The implementation utilizes Python libraries including Pandas, NumPy, and Matplotlib to load and explore the loan dataset. Data preprocessing techniques such as handling missing values, data cleaning, Label Encoding, One-Hot Encoding, Feature Scaling, and Standardization are applied to improve data quality before analysis. Exploratory Data Analysis (EDA) is performed to identify relationships among applicant attributes, and visualizations such as bar charts and histograms are generated to present meaningful insights. Finally, a basic prediction approach is used to estimate the likelihood of loan approval based on the processed data. The project demonstrates how data analysis and visualization can assist financial institutions in making faster and more informed loan approval decisions and provides a foundation for future enhancements using advanced machine learning techniques.

# TABLE OF CONTENTS


| S.No. | Contents | Pages |
|---|---|---|
| 1 | Title Page | i |
| 2 | Certificate | ii |
| 3 | Declaration | iii |
| 4 | Acknowledgement | iv |
| 5 | Abstract | V |
| 6 | Table of Contents | Vi |
| 7 | List of Figures | ix |
| 8 | List of Tables | x |
| **Chapter 1** | **Introduction** | 1 |
| 1.1 | Introduction to the Project | 1 |
| 1.2 | Problem Statement | 1 |
| 1.3 | Objectives of the Project | 1 |
| 1.4 | Scope of the Project | 2 |
| **Chapter 2** | **Dataset Description and Understanding** | 3 |
| 2.1 | Dataset Source | 3 |
| 2.2 | Dataset Description | 3 |
| 2.3 | Features / Attributes | 4 |
| 2.4 | Target Variable, if applicable | 4 |
| 2.5 | Dataset Statistics / Initial Exploration | 5 |
| **Chapter 3** | **Data Preprocessing** | 6 |
| 3.1 | Data Collection / Loading | 6 |
| 3.2 | Handling Missing Values | 6 |
| 3.3 | Handling Duplicate / Inconsistent Data | 7 |
| 3.4 | Feature Scaling | 7 |
| 3.5 | Standardization | 8 |

| | | |
|---|---|---|
| 3.6 | Label Encoding | 8 |
| 3.7 | One-Hot Encoding | 8 |
| 3.8 | Other Project-Specific Preprocessing | 9 |
| **Chapter 4** | **System Design and Methodology** | 10 |
| 4.1 | Proposed Methodology | 10 |
| 4.2 | System Workflow / Architecture | 10 |
| 4.3 | Flowchart | 11 |
| 4.4 | Tools and Technologies Used | 11 |
| 4.5 | Algorithm / Model Used | 12 |
| **Chapter 5** | **Implementation** | 13 |
| 5.1 | Development Environment | 13 |
| 5.2 | Dataset Loading and Exploration | 13 |
| 5.3 | Preprocessing Implementation | 14 |
| 5.4 | Exploratory Data Analysis (EDA) | 14 |
| 5.5 | Model / Analysis Implementation | 15 |
| 5.6 | Important Code Explanation | 16 |
| **Chapter 6** | **Data Visualization and Results** | 18 |
| 6.1 | Bar Graph | 18 |
| 6.2 | Pie Chart | 20 |
| 6.3 | Histogram | 21 |
| 6.4 | Box Plot | 22 |
| 6.5 | Line Chart | 22 |
| 6.6 | Scatter Plot | 23 |
| 6.7 | Project-Specific Visualizations | 23 |
| 6.8 | Interpretation of Visualizations | 25 |
| **Chapter 7** | **Model Evaluation / Analysis of Results** | 26 |
| 7.1 | Evaluation Metrics | 26 |
| 7.2 | Experimental Results | 26 |
| 7.3 | Result Interpretation | 28 |

| | | |
|---|---|---|
| 7.4 | Discussion | 28 |
| **Chapter 8** | **Output Screenshots** | 29 |
| 8.1 | Input / Dataset Screenshots | 29 |
| 8.2 | Preprocessing Outputs | 30 |
| 8.3 | Visualization Outputs | 33 |
| 8.4 | Prediction / Analysis Outputs | 36 |
| **Chapter 9** | **Conclusion and Future Enhancements** | 38 |
| 9.1 | Conclusion | 38 |
| 9.2 | Limitations | 38 |
| 9.3 | Future Enhancements | 38 |
| **Chapter 10** | **References** | 40 |
| **Chapter 11** | **Appendix – Source Code** | 42 |

# LIST OF FIGURES


| Figure No. | Figure Title | Page No. |
|---|---|---|
| Figure 1 | Project Workflow | 11 |
| Figure 2 | Loan Approval Status Distribution | 18 |
| Figure 3 | Gender Distribution | 19 |
| Figure 4 | Education Distribution | 19 |
| Figure 5 | Property Area Distribution | 20 |
| Figure 6 | Loan Approval Status Pie Chart | 20 |
| Figure 7 | Applicant Income Distribution | 21 |
| Figure 8 | Loan Amount Distribution | 21 |
| Figure 9 | Applicant Income Box Plot | 22 |
| Figure 10 | Average Loan Amount by Applicant Income Range | 22 |
| Figure 11 | Applicant Income vs. Loan Amount | 23 |
| Figure 12 | Credit History vs. Loan Status | 23 |
| Figure 13 | Education vs. Loan Status | 24 |
| Figure 14 | Property Area vs. Loan Status | 24 |
| Figure 15 | Correlation Matrix | 25 |
| Figure 16 | Confusion Matrix | 27 |

# LIST OF TABLES


| Table No. | Title | Page No. |
|---|---|---|
| **Table 1** | Dataset Summary | 3 |
| **Table 2** | Dataset Attributes | 4 |
| **Table 3** | Missing Values Before Cleaning | 6 |
| **Table 4** | Proposed System Workflow | 10 |
| **Table 5** | Tools and Technologies Used | 11 |
| **Table 6** | Dataset Exploration Results | 14 |
| **Table 7** | Exploratory Data Analysis Results | 15 |
| **Table 8** | Selected Model Features | 16 |
| **Table 9** | Model Evaluation Results | 27 |
| **Table 10** | Classification Report | 28 |

# CHAPTER 1

# INTRODUCTION

## 1.1 Introduction to the Project

The **Loan Prediction Data Analysis System** is developed to analyze loan applicant data and identify the factors associated with loan approval. Banks receive a large number of loan applications containing different applicant and financial details. Analyzing this information helps in understanding patterns related to loan approval.

This project uses Python-based data analysis techniques to clean, preprocess, analyze, and visualize loan applicant data. A basic prediction model is also used to estimate loan approval based on the processed data.

## 1.2 Problem Statement

Banks receive a large number of loan applications, making manual evaluation time-consuming. Loan approval depends on several applicant and financial factors such as income, credit history, loan amount, loan term, and property area.

The project focuses on analyzing these factors to identify their relationship with loan approval and to perform basic loan approval prediction using the available data.

## 1.3 Objectives of the Project

The main objectives of the project are:

- To analyse and understand the loan applicant dataset.
- To clean and preprocess the collected data.
- To identify important factors associated with loan approval.
- To perform Exploratory Data Analysis (EDA).
- To visualize patterns and relationships in the data.
- To perform basic loan approval prediction using the processed data.

## 1.4 Scope of the Project

The project focuses on analyzing loan applicant data using data preprocessing, exploratory data analysis, visualization, and basic prediction techniques.

The system provides insights into applicant attributes and their relationship with loan approval. It also provides a foundation for developing more advanced loan prediction systems in the future.

# CHAPTER 2

# DATASET DESCRIPTION AND UNDERSTANDING

## 2.1 Dataset Source

The dataset used for this project is a **Loan Prediction dataset in CSV format**, obtained from **Kaggle**. It contains information about loan applicants and their financial and personal attributes.

The dataset is selected to analyze the factors associated with loan approval and to perform basic loan approval prediction.

## 2.2 Dataset Description

The dataset contains **614 records and 13 attributes**. It includes both categorical and numerical information related to loan applicants.

The dataset contains applicant details such as gender, marital status, dependents, education, employment status, income, loan amount, loan term, credit history, and property area. The **Loan_Status** attribute represents the loan outcome.

The dataset contains missing values, which makes it suitable for demonstrating data cleaning and preprocessing techniques.

## Table 1. Dataset Summary

| Parameter | Details |
|---|---|
| Dataset | Loan Prediction Dataset |
| File Format | CSV |
| Number of Records | 614 |
| Number of Attributes | 13 |
| Data Types | Numerical and Categorical |

| | |
|---|---|
| Target Variable | Loan_Status |
| Duplicate Records | 0 |

## 2.3 Feature/Attributes

The dataset contains the following attributes:

## Table 2. Dataset Attributes

| Attribute | Description |
|---|---|
| Loan_ID | Unique identifier for the loan application |
| Gender | Gender of the applicant |
| Married | Marital status of the applicant |
| Dependents | Number of dependents |
| Education | Education level of the applicant |
| Self_Employed | Employment status of the applicant |
| ApplicantIncome | Income of the applicant |
| CoapplicantIncome | Income of the co-applicant |
| LoanAmount | Requested loan amount |
| Loan_Amount_Term | Loan repayment period |
| Credit_History | Credit history indicator |
| Property_Area | Location category of the property |
| Loan_Status | Loan approval status |

## 2.4 Target Variable

The target variable of the project is **Loan_Status**.

It represents the outcome of the loan application:

- **Y** – Loan Approved
- **N** – Loan Not Approved

The target variable is converted into numerical form during preprocessing so that it can be used by the prediction model.

## 2.5 Dataset Statistics / Initial Exploration

Initial exploration is performed to understand the structure and quality of the dataset. The dataset contains **614 records and 13 attributes**.

The initial analysis includes checking the dataset shape, column names, data types, statistical information, missing values, and duplicate records.

The dataset contains **149 missing values** before preprocessing, while **no duplicate records** are found.

The distribution of the target variable is:

- **Loan Approved (Y): 422**
- **Loan Not Approved (N): 192**

# CHAPTER 3

# DATA PREPROCESSING

## 3.1 Data Collection / Loading

The loan dataset is stored in CSV format and is loaded into Python using the **Pandas** library.

The dataset is converted into a Pandas DataFrame for further analysis and preprocessing.

The dataset shape is checked after loading to verify the number of records and attributes.

**Code:**

```
import pandas as pd
df = pd.read_csv("loan.csv")
print("Number of rows and columns:", df.shape)
```

**Output:**

The dataset contains **614 rows and 13 columns**.

## 3.2 Handling Missing Values

Missing values are checked using the isnull() function. The dataset initially contains **149 missing values**.

Missing values are handled using suitable statistical methods:

- **Categorical attributes** are filled using the **mode**.
- **Numerical attributes** are filled using the **median**.

## Table 3. Missing Values Before Cleaning

| Attribute | Missing Values | Method Used |
|---|---|---|
| Gender | 13 | Mode |
| Married | 3 | Mode |
| Dependents | 15 | Mode |
| Self_Employed | 32 | Mode |
| LoanAmount | 22 | Median |
| Loan_Amount_Term | 14 | Median |
| Credit_History | 50 | Median |

## 3.3 Handling Duplicate / Inconsistent Data

Duplicate records are checked using the duplicated() function.

The dataset contains **0 duplicate rows**.

The Dependents attribute contains the value **"3+"**, which is converted into the numerical value **3** to make it suitable for model processing.

**Code**

```
print("Number of Duplicate Rows:", df.duplicated().sum())
model_df["Dependents"] = model_df["Dependents"].replace("3+", 3)
model_df["Dependents"] = model_df["Dependents"].astype(int)
```

## 3.4 Feature Scaling

Feature Scaling is applied to numerical attributes so that their values are brought to a comparable scale.

**Min-Max Scaling** is applied to:

- ApplicantIncome
- CoapplicantIncome
- LoanAmount
- Loan_Amount_Term

The MinMaxScaler from Scikit-learn is used for this process.

**Code:**

```
features_to_scale = [
    "ApplicantIncome",
    "CoapplicantIncome",
    "LoanAmount",
    "Loan_Amount_Term"
]
scaler = MinMaxScaler()
model_df[features_to_scale] = scaler.fit_transform(
    model_df[features_to_scale]
)
```

## 3.5 Standardization

Standardization is applied to the selected numerical attributes using **StandardScaler**.

It transforms the numerical features based on their mean and standard deviation, making them suitable for further analysis and model processing.

## 3.6 Label Encoding

Label Encoding is used to convert categorical values into numerical values.

The following attributes are encoded:

- Gender
- Married
- Education
- Self_Employed
- Loan_Status

**Code:**

```
le = LabelEncoder()
model_df["Gender"] = le.fit_transform(model_df["Gender"])
model_df["Married"] = le.fit_transform(model_df["Married"])
model_df["Education"] = le.fit_transform(model_df["Education"])
model_df["Self_Employed"] = le.fit_transform(model_df["Self_Employed"])
model_df["Loan_Status"] = le.fit_transform(model_df["Loan_Status"])
```

## 3.7 One-Hot Encoding

**One-Hot Encoding** is applied to the Property_Area attribute because it contains multiple categories.

The categories are converted into separate binary columns.

**Code:**

```
model_df = pd.get_dummies(
    model_df,
    columns=["Property_Area"],
    drop_first=True
```

)

This produces encoded attributes such as:

- Property_Area_Semiurban
- Property_Area_Urban

## 3.8 Other Project-Specific Preprocessing

Before model training, Loan_ID is removed because it is an identifier and does not provide useful predictive information.

The Boolean values generated during One-Hot Encoding are converted into integer values for model processing.

After completing the preprocessing steps, the dataset is ready for **Exploratory Data Analysis, feature selection, model training, and prediction**.

# CHAPTER 4

# SYSTEM DESIGN AND METHODOLOGY

## 4.1 Proposed Methodology

The proposed methodology follows a sequence of data analysis and prediction steps. The dataset is first loaded and examined for its structure and data quality. Missing values and duplicate records are handled during preprocessing.

The cleaned data is then transformed using encoding, feature scaling, and standardization techniques. Exploratory Data Analysis (EDA) is performed to identify patterns and relationships among the attributes. After feature preparation, the data is divided into training and testing sets. A **Logistic Regression** model is then trained to perform basic loan approval prediction.

## 4.2 System Workflow / Architecture

The system follows the workflow:

**Data Collection → Data Import → Data Cleaning → Data Preprocessing → EDA → Data Visualization → Basic Loan Prediction → Model Evaluation**

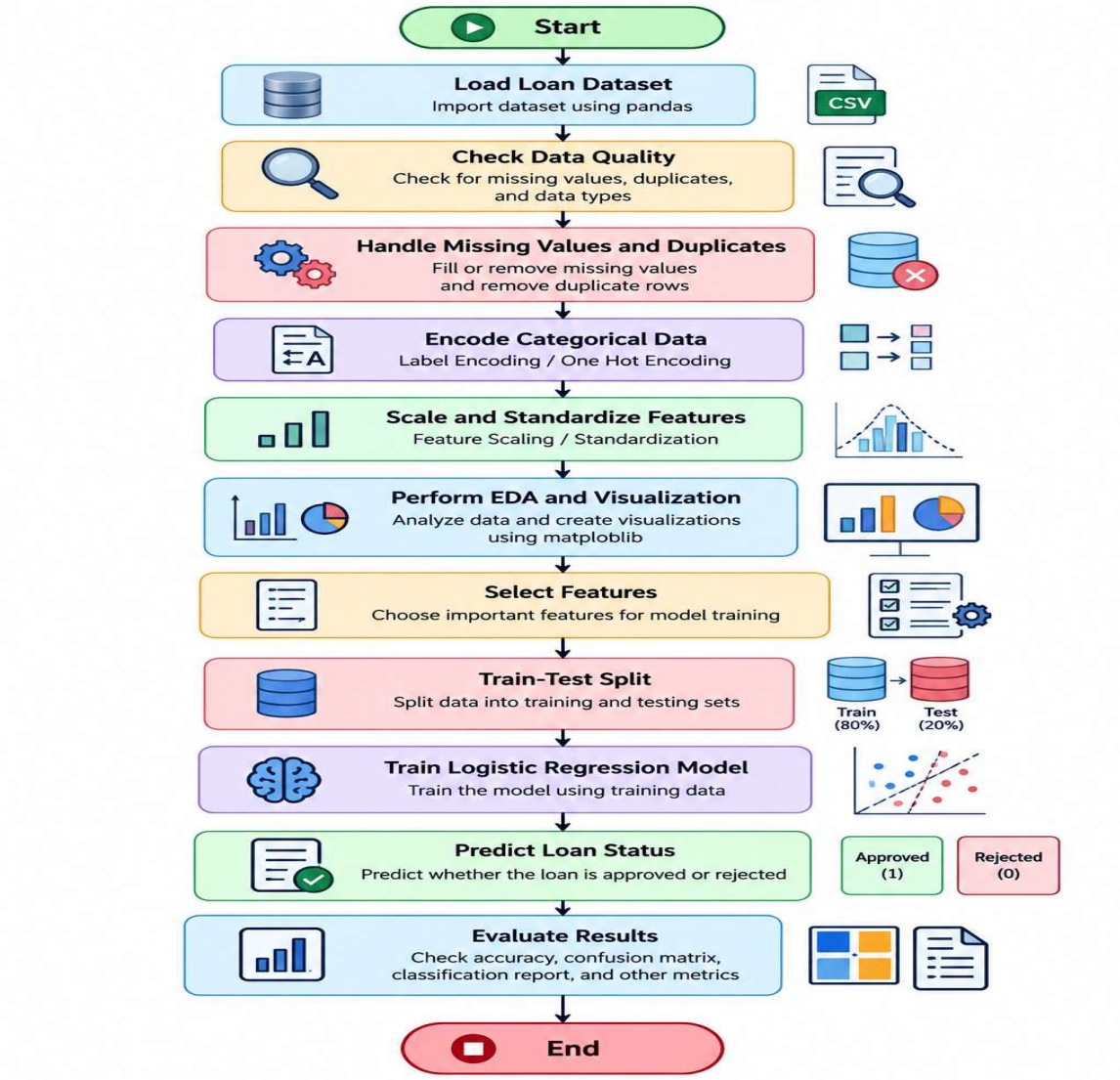
Each stage contributes to preparing the data and obtaining meaningful results from the loan applicant dataset.

## Table 4. Proposed System Workflow

| Step | Stage | Description |
|---|---|---|
| 1 | Data Collection | Obtain the loan dataset in CSV format |
| 2 | Data Import | Load the dataset using Pandas |
| 3 | Data Cleaning | Check and handle missing values and duplicates |
| 4 | Data Preprocessing | Apply encoding, scaling, and standardization |
| 5 | EDA | Analyze patterns and relationships |
| 6 | Data Visualization | Represent findings using charts and graphs |
| 7 | Loan Prediction | Predict the loan approval outcome |

## 4.3 Flowchart

**Figure 1. Project Workflow**




## 4.4 Tools and Technologies Used

The following tools and technologies are used in the project:

## Table 5. Tools and Technologies Used

| Tool / Technology | Purpose |
|---|---|
| Python | Project implementation |

| | |
|---|---|
| Pandas | Data loading, cleaning, and analysis |
| NumPy | Numerical operations |
| Matplotlib | Data visualization |
| Scikit-learn | Preprocessing, model training, and evaluation |
| Google Colab | Development and execution environment |
| CSV | Dataset storage format |

## 4.5 Algorithm / Model Used

**Logistic Regression** is used as the classification algorithm for basic loan approval prediction.

The preprocessed applicant attributes are provided as input to the model, while **Loan_Status** is used as the target variable. The model learns the relationship between the input features and the loan approval outcome and then predicts the outcome for unseen test data.

The model is trained using an **80:20 train-test split** with stratification.

# CHAPTER 5

# IMPLEMENTATION

## 5.1 Development Environment

The project is implemented using **Python** in **Google Colab**. The following libraries are used:

- **Pandas** for data loading and data analysis.
- **NumPy** for numerical operations.
- **Matplotlib** for data visualization.
- **Scikit-learn** for preprocessing, model training, and evaluation.

## 5.2 Dataset Loading and Exploration

The dataset is loaded using Pandas and stored in a DataFrame.

**Code:**

```
import pandas as pd
df = pd.read_csv("loan.csv")
print(df.head())
print(df.shape)
print(df.info())
print(df.describe())
```

The initial exploration confirms that the dataset contains **614 records and 13 attributes**.

The following checks are performed:

- Dataset shape
- First few records
- Column names
- Data types
- Statistical summary
- Missing values

- Duplicate records

## Table 6. Dataset Exploration Results

| Parameter | Result |
|---|---|
| Dataset Shape | 614 × 13 |
| Duplicate Rows | 0 |
| Total Missing Values | 149 |
| Loan Approved (Y) | 422 |
| Loan Not Approved (N) | 192 |

## 5.3 Preprocessing Implementation

The data is preprocessed before performing analysis and prediction.

The following preprocessing techniques are implemented:

1. Missing values are handled using mode and median.
2. Duplicate records are checked.
3. 3+ in the Dependents attribute is converted to 3.
4. Categorical attributes are converted into numerical values using Label Encoding.
5. Property Area is converted using One-Hot Encoding.
6. Numerical attributes are scaled using Min-Max Scaling.
7. Standardization is applied to selected numerical attributes.
8. Loan_ID is removed before model training.

After preprocessing, the data is prepared for EDA and model training.

## 5.4 Exploratory Data Analysis (EDA)

Exploratory Data Analysis is performed to understand the distribution of important attributes and their relationship with loan approval.

The analysis includes:

- Loan Status distribution
- Gender distribution

- Education distribution
- Property Area distribution
- Applicant Income distribution
- Loan Amount distribution
- Credit History vs. Loan Status
- Education vs. Loan Status
- Property Area vs. Loan Status
- Correlation analysis

## Table 7. Exploratory Data Analysis Results

| Attribute | Result |
|---|---|
| Loan Status | Y: 422, N: 192 |
| Gender | Male: 502, Female: 112 |
| Education | Graduate: 480, Not Graduate: 134 |
| Property Area | Semiurban: 233, Urban: 202, Rural: 179 |

## 5.5 Model / Analysis Implementation

After preprocessing, the following features are selected for model training:

- Gender
- Married
- Dependents
- Education
- Self_Employed
- ApplicantIncome
- CoapplicantIncome
- LoanAmount
- Loan_Amount_Term
- Credit_History

- Property_Area_Semiurban
- Property_Area_Urban

The target variable is **Loan_Status**.

The dataset is divided into training and testing data using an **80:20 stratified train-test split**.

- Training records: **491**
- Testing records: **123**

A **Logistic Regression** model is then trained using the training data.

## Table 8. Selected Model Features

| Category | Details |
|---|---|
| Input Features | 12 processed attributes |
| Target Variable | Loan_Status |
| Training Data | 491 records |
| Testing Data | 123 records |
| Model | Logistic Regression |
| Split | 80:20 Stratified Split |

## 5.6 Important Code Explanation

The main implementation steps are:

**Code:**

```
X = model_df[features]
y = model_df["Loan_Status"]
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.20,
    random_state=42,
    stratify=y
)
model = LogisticRegression(max_iter=1000)
model.fit(X_train, y_train)
```

```
y_pred = model.predict(X_test)
```

The trained model generates predictions for the test data. These predictions are evaluated using accuracy, classification report, and confusion matrix.

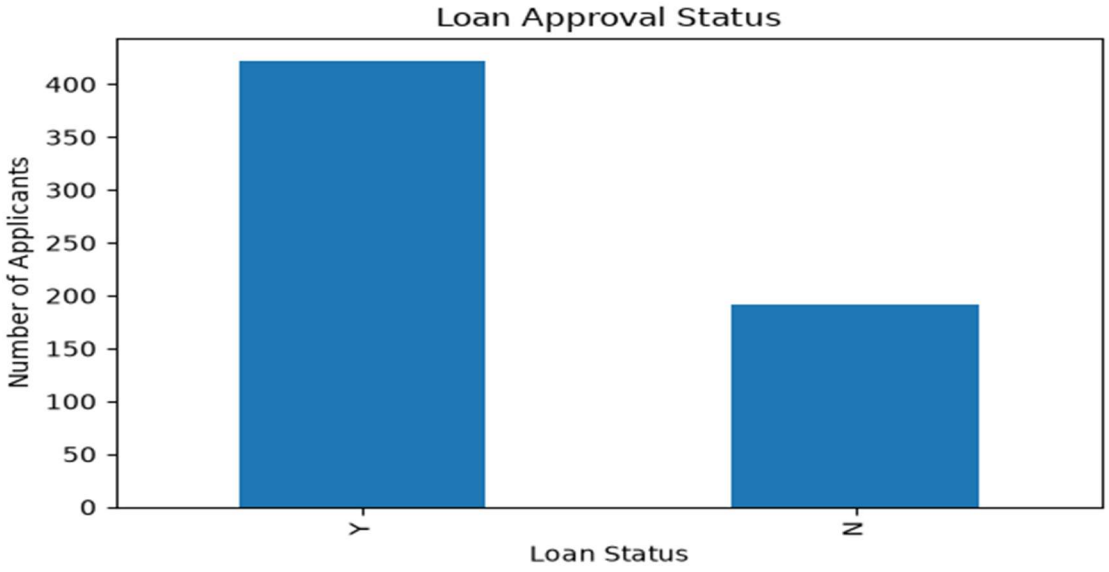
# CHAPTER 6

# DATA VISUALIZATION AND RESULTS

## 6.1 Bar Graph

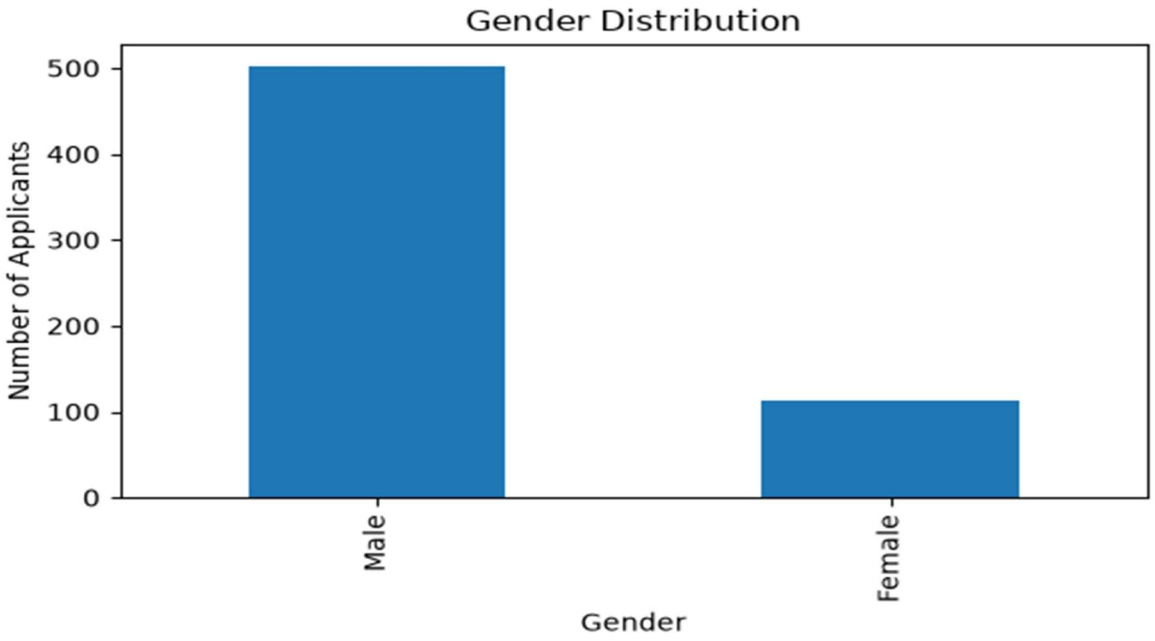
Bar graphs are used to represent categorical distributions and compare loan-related categories. The project uses bar graphs for loan approval status, gender, education, and property area.

**Figure 2. Loan Approval Status Distribution**



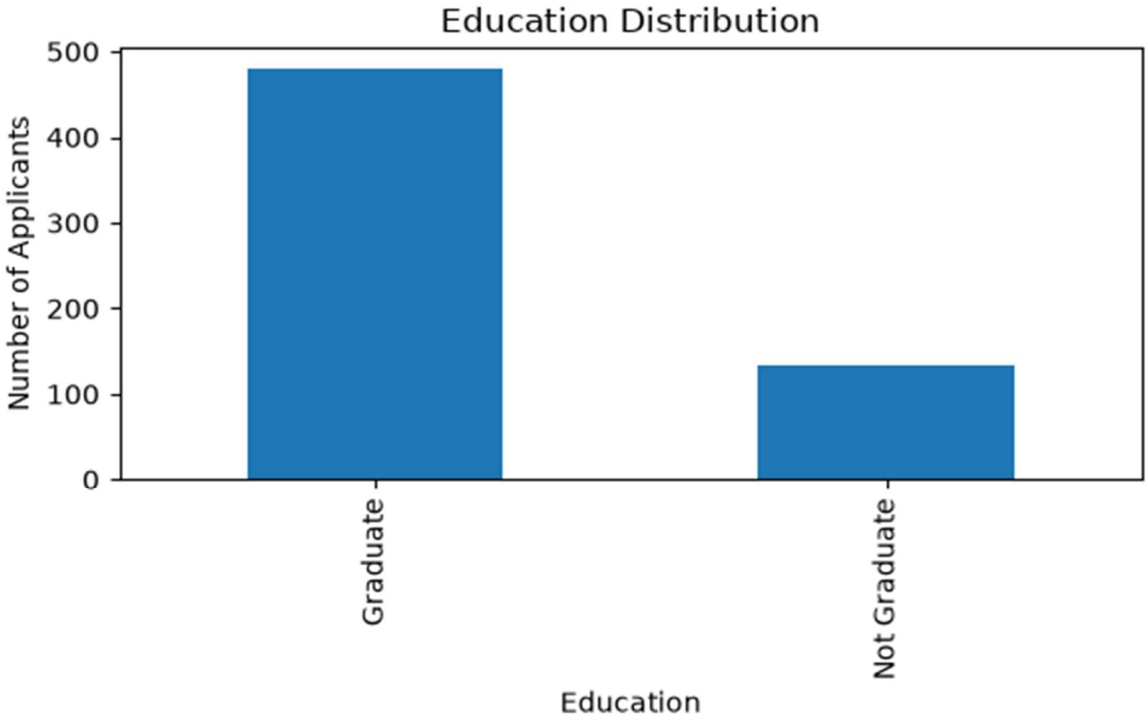

The graph shows the distribution of approved and not-approved loan applications. The dataset contains **422 approved (Y)** applications and **192 not-approved (N)** applications.

**Figure 3. Gender Distribution**



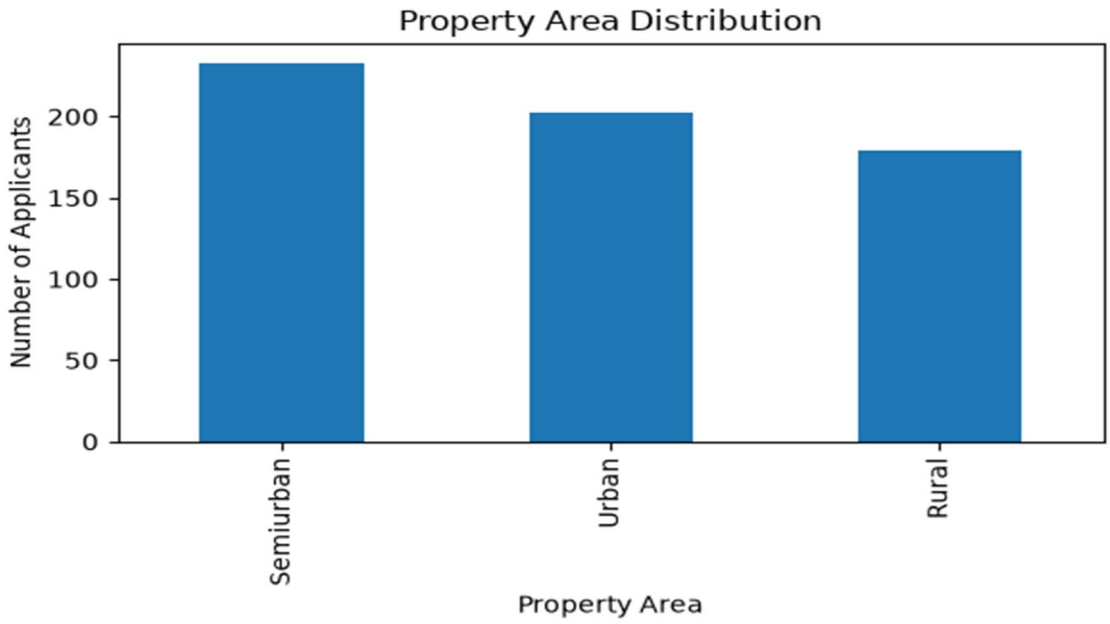

The graph shows **502 male** applicants and **112 female** applicants in the dataset.

**Figure 4. Education Distribution**




The graph shows **480 Graduate** applicants and **134 Not Graduate** applicants.

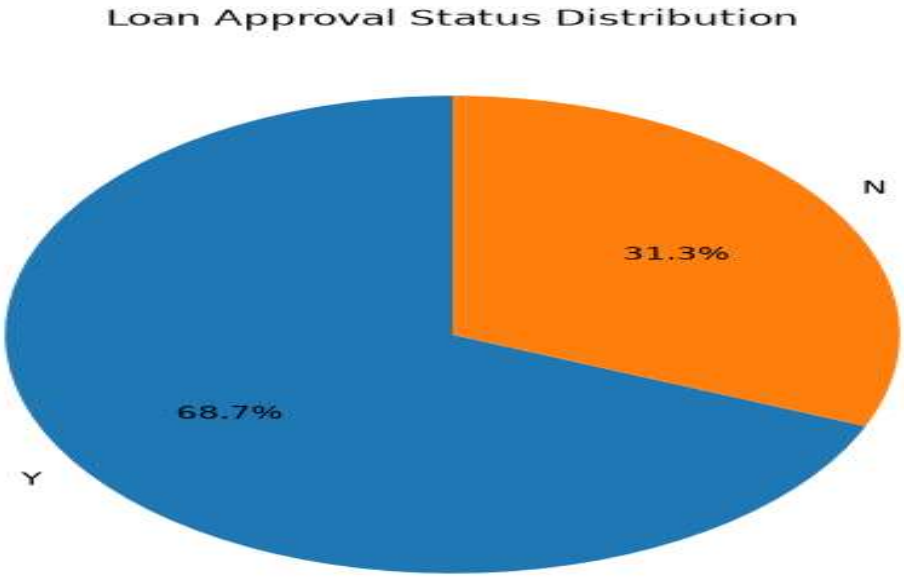
**Figure 5. Property Area Distribution**




The graph represents applicants according to property area. The counts are **233 Semiurban**, **202 Urban**, and **179 Rural**.

## 6.2 Pie Chart

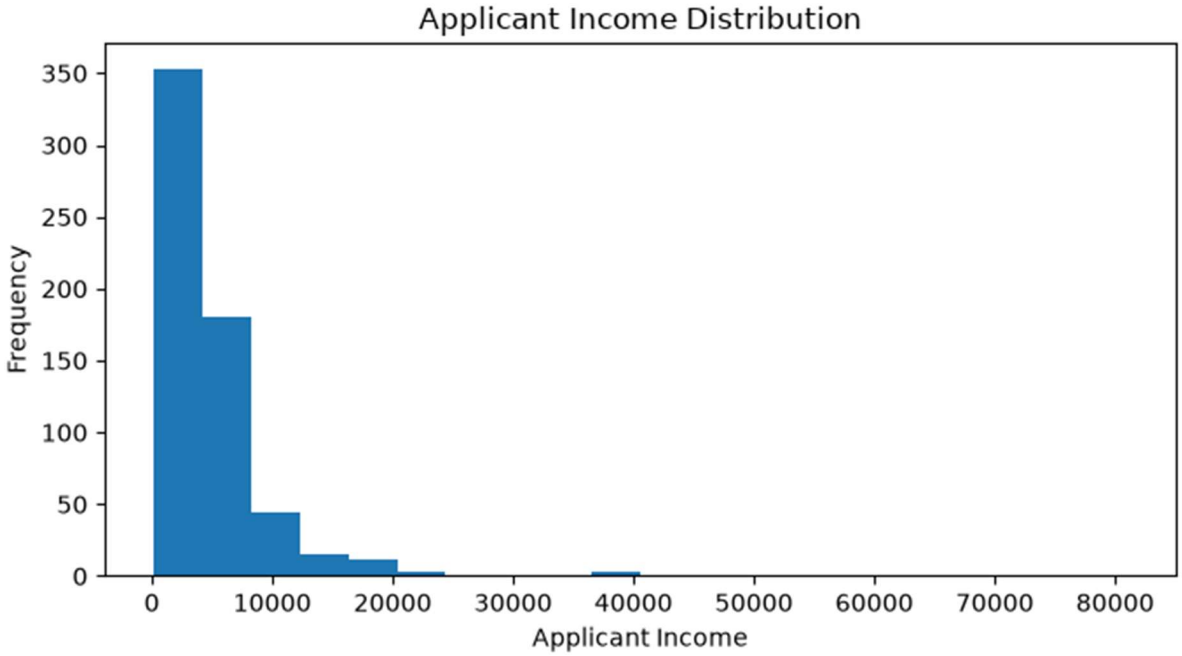
**Figure 6. Loan Approval Status – Pie Chart**



## 6.3 Histogram

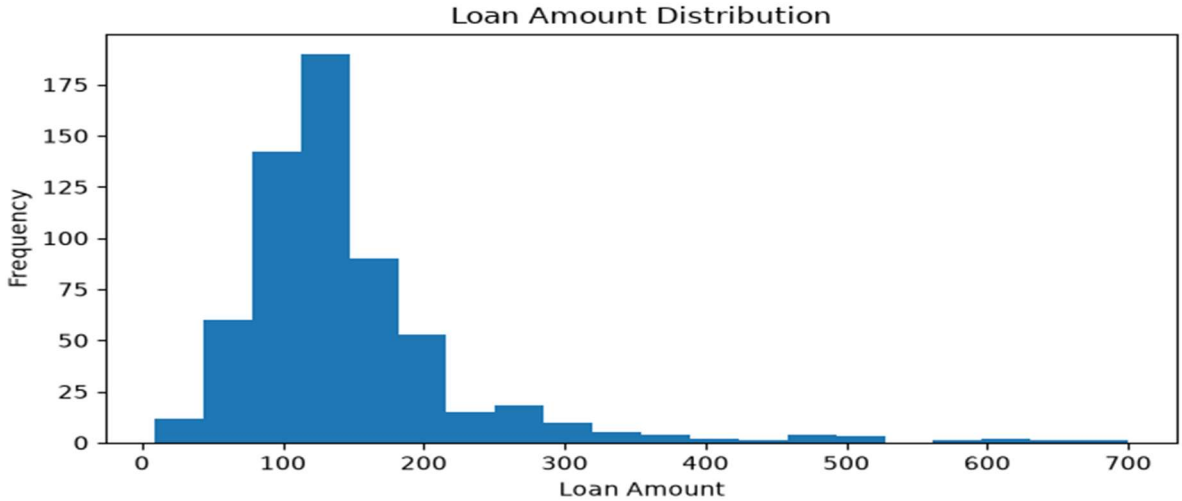
Histograms are used to study the distribution of numerical attributes.

**Figure 7. Applicant Income Distribution**




The histogram represents the frequency distribution of applicant income values.

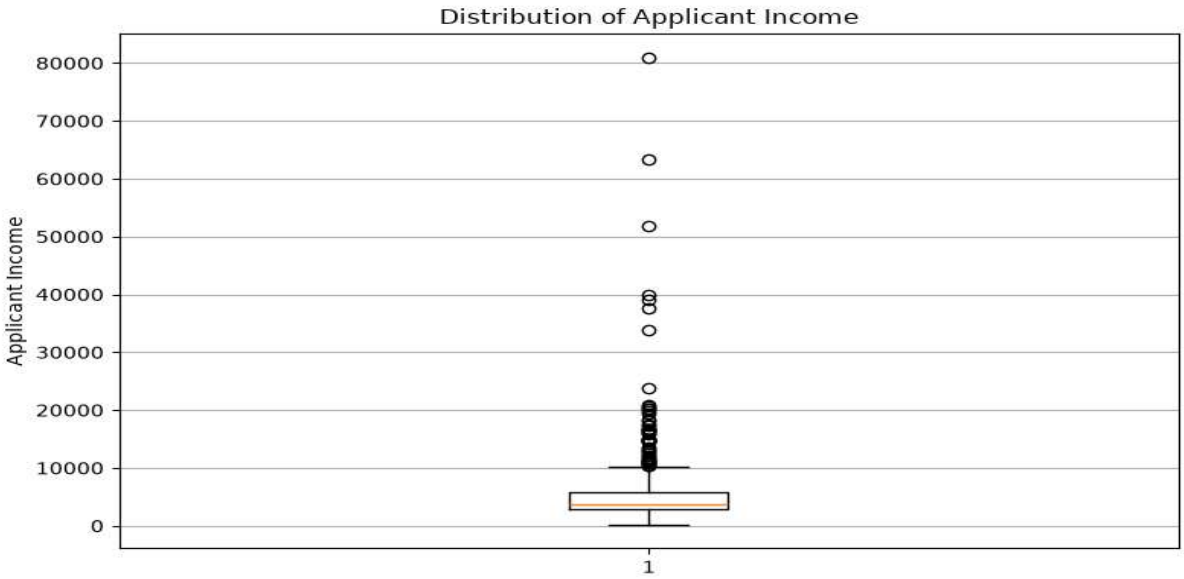
**Figure 8. Loan Amount Distribution**




The histogram represents the frequency distribution of requested loan amounts.

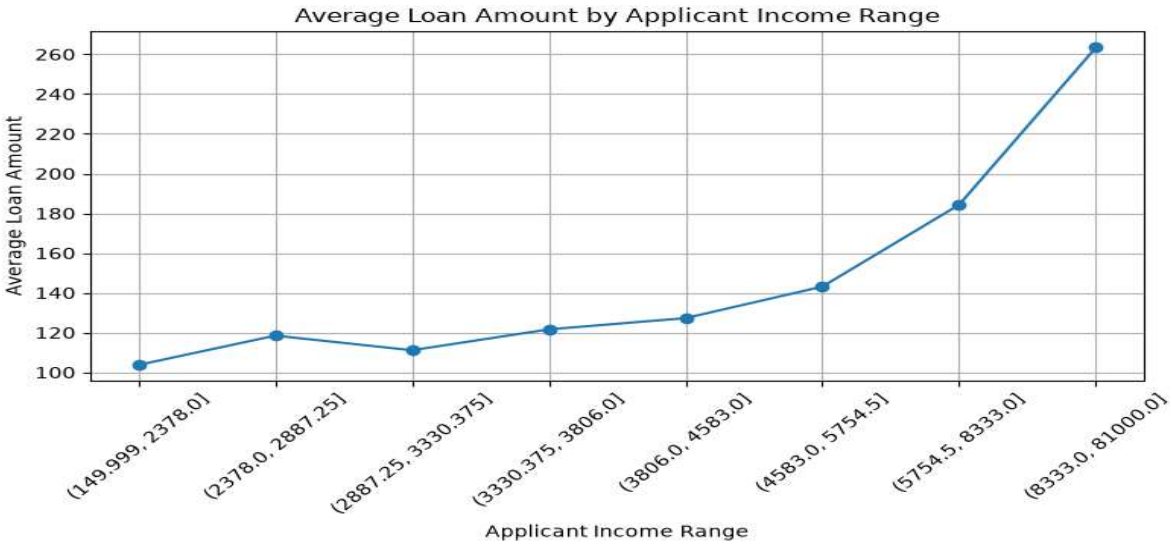
## 6.4 Box Plot

**Figure 9. Average Income – Box Plot**



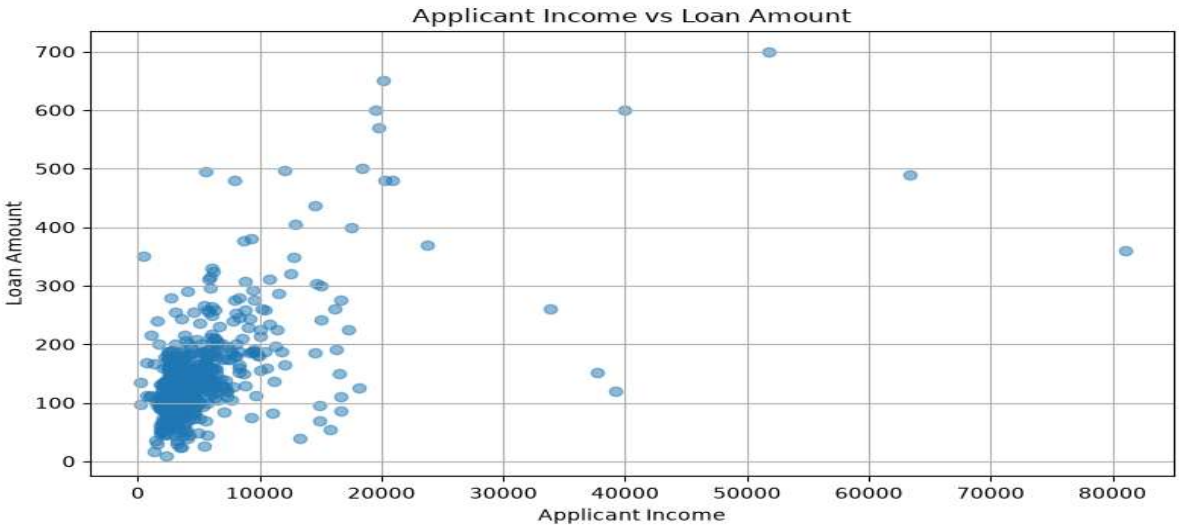

## 6.5 Line Chart

**Figure 10. Average Loan Amount by Applicant Income Range**




## 6.6 Scatter Plot

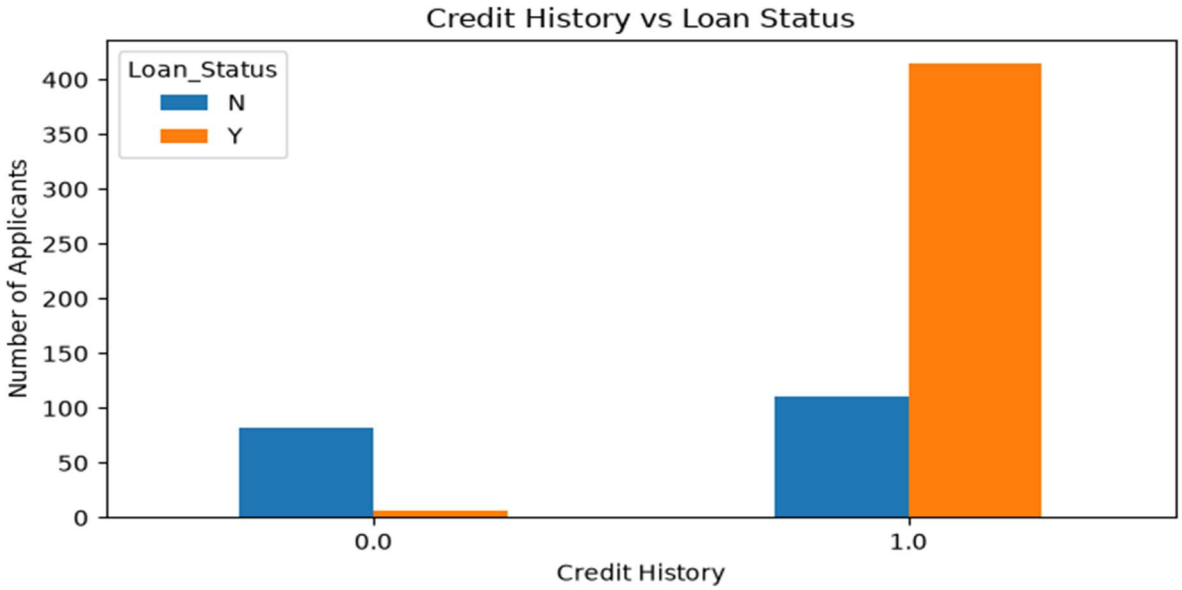
**Figure 11. Applicant Income vs Loan Amount**




## 6.7 Project-Specific Visualizations

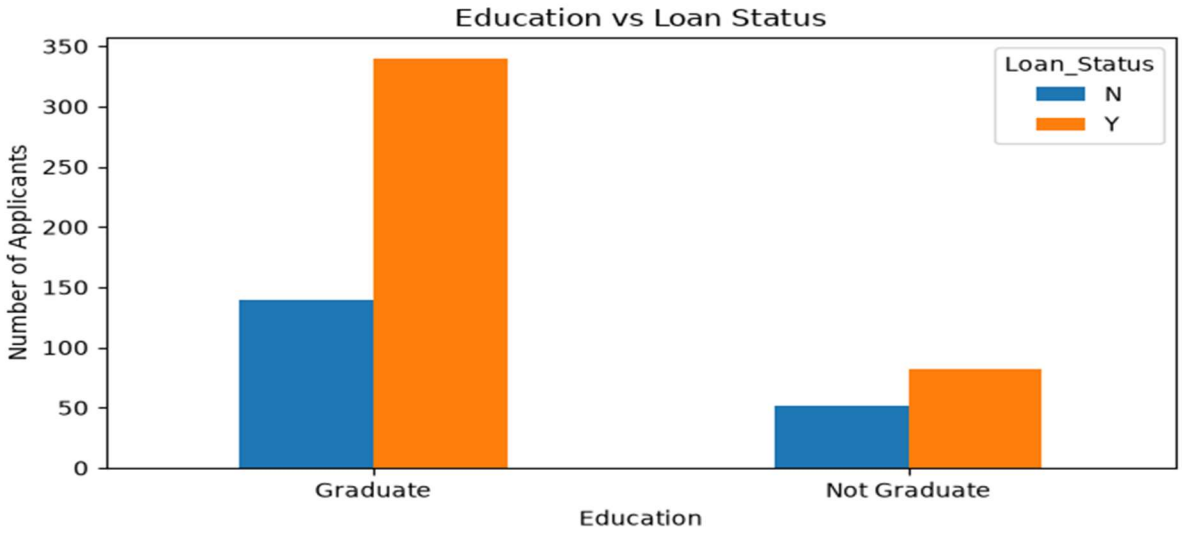
The project includes visualizations that examine relationships between applicant attributes and loan status.

**Figure 12. Credit History vs. Loan Status**



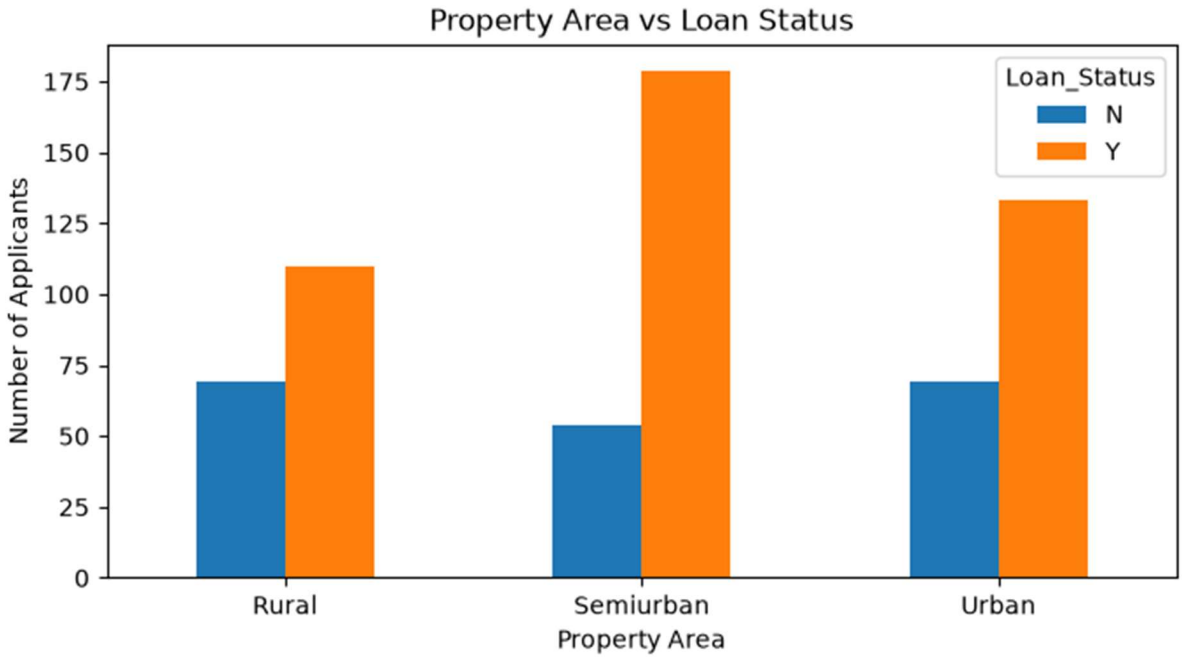

This visualization compares loan approval outcomes for different credit-history values and helps examine the relationship between credit history and loan status.

**Figure 13. Education vs. Loan Status**



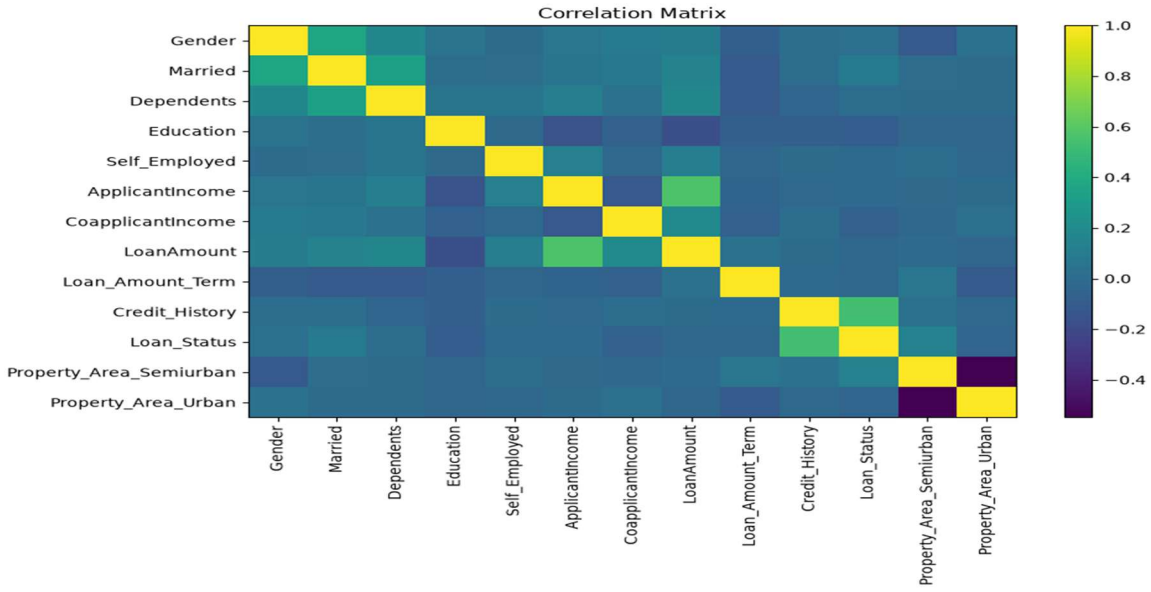

This visualization compares loan approval outcomes across Graduate and Not Graduate applicants.

**Figure 14. Property Area vs. Loan Status**




This visualization compares loan approval outcomes across Rural, Semiurban, and Urban property areas.

**Figure 15. Correlation Matrix**




The correlation matrix is used to examine the relationships among the numerical and encoded variables used during analysis. The notebook includes correlation analysis as part of the visualization stage.

## 6.8 Interpretation of Visualizations

- The dataset contains **422 approved applications and 192 not-approved applications**.
- The dataset contains more **male applicants than female applicants**.
- **Graduate applicants** form the larger education category.
- **Semiurban property areas** contain the highest number of records among the three property-area categories.
- Applicant income and loan amount are examined using histograms to understand their distributions.
- Credit history is analyzed against loan status to examine its relationship with approval outcomes.
- Education and property area are also compared with loan status to identify patterns in the dataset.
- The correlation matrix provides an overall view of relationships among the analysed variables.

# CHAPTER 7

# MODEL EVALUATION / ANALYSIS OF RESULTS

This chapter presents the evaluation of the Logistic Regression model used for basic loan approval prediction. The model was evaluated using the test dataset and standard classification metrics.

## 7.1 Evaluation Metrics

The following metrics were used to evaluate the prediction model:

**Accuracy**

Accuracy represents the proportion of correctly predicted loan outcomes among all test records.

**Precision**

Precision indicates how many of the records predicted as a particular class were actually members of that class.

**Recall**

Recall indicates how many actual records of a particular class were correctly identified by the model.

**F1-Score**

F1-score provides a combined measure of precision and recall.

**Confusion Matrix**

A confusion matrix shows the number of correct and incorrect predictions for each class.

## 7.2 Experimental Results

The dataset was divided into training and testing sets using an **80:20 stratified split**.

- Training records: **491**
- Testing records: **123**
- Model used: **Logistic Regression**
- Accuracy: **85.37%**

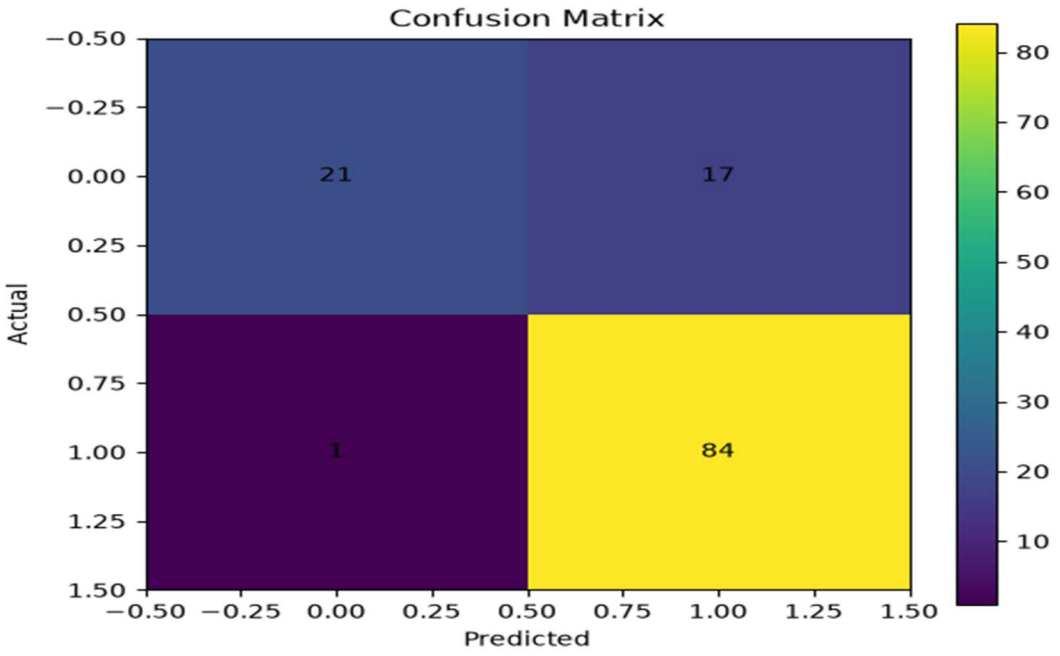
## Table 9. Model Evaluation Results

| Parameter | Result |
|---|---|
| Model | Logistic Regression |
| Training Records | 491 |
| Testing Records | 123 |
| Train-Test Split | 80:20 |
| Accuracy | 85.37% |
| Confusion Matrix | [[21, 17], [1, 84]] |

The confusion matrix obtained from the model is:

- True Negative = 21
- False Positive = 17
- False Negative = 1
- True Positive = 84

**Figure 16. Confusion Matrix**



## Table 10. Classification Report

| Class | Precision | Recall | F1-Score | Support |
|---|---|---|---|---|
| 0 | 0.95 | 0.55 | 0.70 | 38 |
| 1 | 0.83 | 0.99 | 0.90 | 85 |
| Macro Average | 0.89 | 0.77 | 0.80 | 123 |
| Weighted Average | 0.87 | 0.85 | 0.84 | 123 |

## 7.3 Result Interpretation

The Logistic Regression model achieved an accuracy of **85.37%** on the test dataset.

The confusion matrix shows that the model correctly classified **21 records of class 0** and **84 records of class 1**. It incorrectly classified **17 records as class 1 when they belonged to class 0**, while only **1 record of class 1 was classified as class 0**.

The classification report shows a recall of **0.99 for class 1**, while class 0 has a recall of **0.55**. This indicates that the model identified class 1 records more frequently than class 0 records in this particular test set.

## 7.4 Discussion

The results demonstrate that the processed loan dataset can be used with Logistic Regression for basic loan-status prediction. The model achieved **85.37% accuracy** on the selected test set.

The results are specific to the dataset, preprocessing techniques, selected features, and train-test split used in this project. Therefore, the model should be considered a **basic analytical prediction model**, rather than a complete real-world banking loan approval system.

# CHAPTER 8

# OUTPUT SCREENSHOTS

This chapter presents selected screenshots of the major outputs obtained during the implementation of the **Loan Prediction Data Analysis System**. The screenshots provide visual evidence of dataset loading, preprocessing, visualization, model evaluation, and prediction.

## 8.1 Input / Dataset Screenshots

The dataset was loaded from a CSV file using Pandas. Initial exploration was performed to understand the structure, dimensions, attributes, and data quality of the dataset.

**Code:**

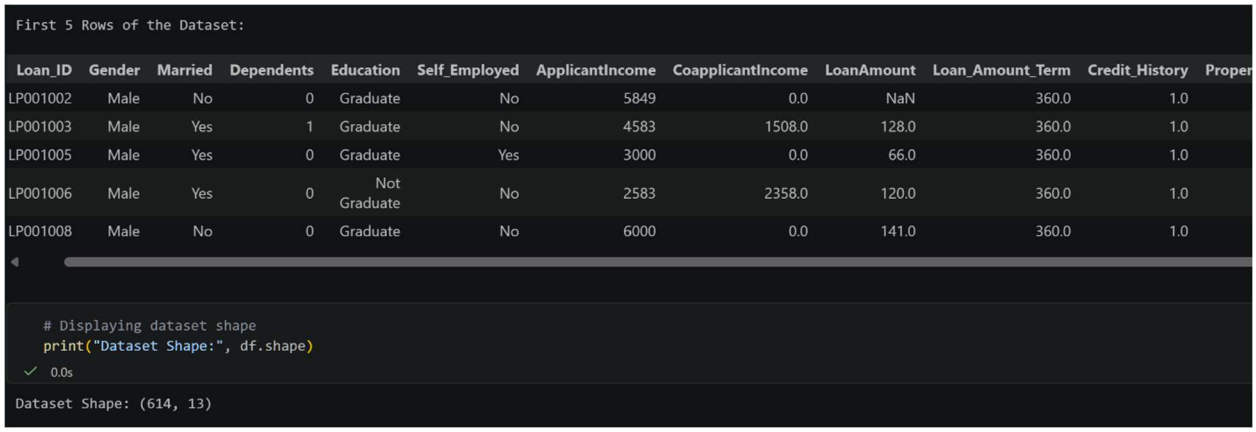
```
# 2. LOADING THE DATASET
df = pd.read_csv("loan.csv")
print("Dataset loaded successfully.")
print("Number of rows and columns:", df.shape)

# Displaying first 5 rows
print("First 5 Rows of the Dataset:")
display(df.head())
# Displaying dataset shape
print("Dataset Shape:", df.shape)
```

**Output:**

```
Dataset loaded successfully.
Number of rows and columns: (614, 13)
Dataset Shape: (614, 13)
```

***Image 8.1 Depicts*** *Dataset Loading and Initial Exploration*




The image contain:

- pd.read_csv()
- df.head()
- df.shape
- df.info()

## 8.2 Preprocessing Outputs

The dataset was preprocessed before performing analysis and prediction. Missing values were handled, duplicate records were checked, the Dependents attribute was cleaned, and categorical attributes were encoded.

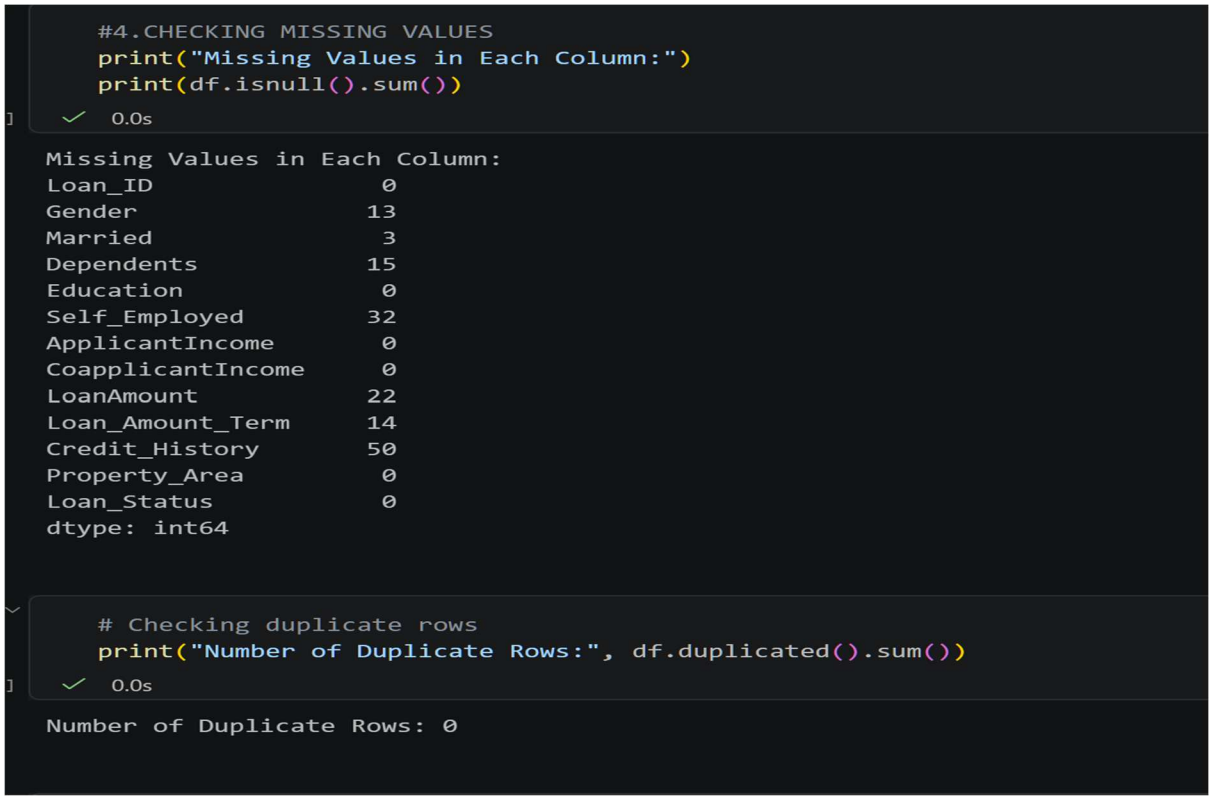
**Code:**

```
#4.CHECKING MISSING VALUES
print("Missing Values in Each Column:")
print(df.isnull().sum())
# Checking duplicate rows
print("Number of Duplicate Rows:", df.duplicated().sum())
```

**Output:**

Number of Duplicate Rows: 0

***Image 8.2 Depicts*** *Data Preprocessing and Encoding*




The image show the relevant preprocessing code and output for:

- Missing-value handling
- Duplicate checking
- Conversion of 3+ to 3
- Label Encoding
- One-Hot Encoding

**Feature Scaling and Standardization**

Numerical attributes were processed using Min-Max Scaling and Standardization to prepare the data for analysis and model training.

**Code:**

#11. FEATURE SCALING

features_to_scale = [

"ApplicantIncome",

"CoapplicantIncome",

"LoanAmount",

"Loan_Amount_Term"

```
]
scaler = MinMaxScaler()
model_df[features_to_scale] = scaler.fit_transform(
model_df[features_to_scale]
)

print("Feature Scaling using Min-Max Scaling completed.")
display(model_df[features_to_scale].head())

#12. STANDARDIZATION
standard_scaler = StandardScaler()
standardized_data = standard_scaler.fit_transform(
df[
    [
     "ApplicantIncome",
     "CoapplicantIncome",
     "LoanAmount",
     "Loan_Amount_Term"
    ]
  ]
)
standardized_df = pd.DataFrame(
standardized_data,
columns=[
    "ApplicantIncome",
    "CoapplicantIncome",
    "LoanAmount",
    "Loan_Amount_Term"
   ]
)
print("Standardization completed.")
```

display(standardized_df.head())

**Output:**

***Image 8.3 depicts*** *Feature Scaling and Standardization*




The screenshot should show the implementation of:

- Min-Max Scaling
- Standardization using StandardScaler

## 8.3 Visualization Outputs

The project includes several visualizations to understand the distribution of applicant data and relationships between different attributes and loan approval.

**Code:**

#LOAN APPROVAL STATUS

plt.figure(figsize=(6, 4))

df["Loan_Status"].value_counts().plot(kind="bar")

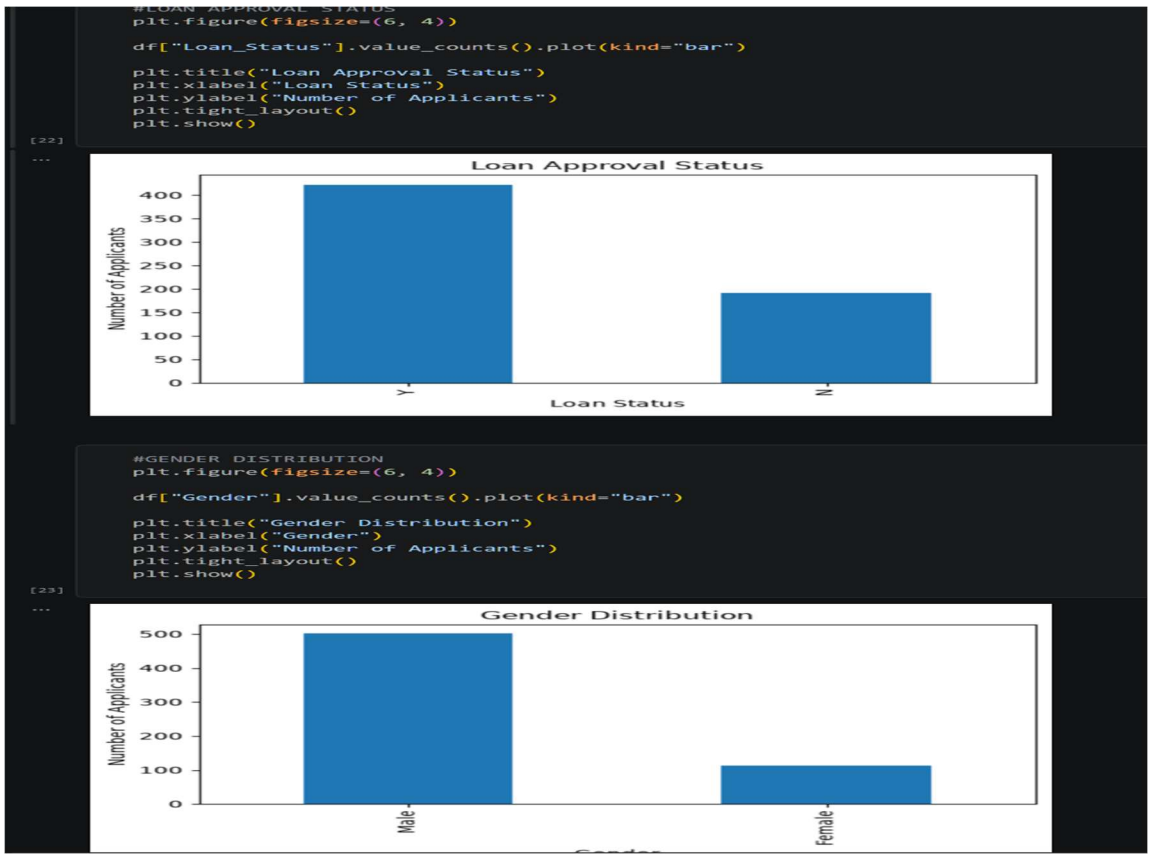
plt.title("Loan Approval Status")

```
plt.xlabel("Loan Status")
plt.ylabel("Number of Applicants")
plt.tight_layout()
plt.show()

#GENDER DISTRIBUTION
plt.figure(figsize=(6, 4))
df["Gender"].value_counts().plot(kind="bar")
plt.title("Gender Distribution")
plt.xlabel("Gender")
plt.ylabel("Number of Applicants")
plt.tight_layout()
plt.show()
```

**Output:**

***Image 8.4 Depicts*** *Exploratory Data Analysis Visualizations*



Include a combined image containing selected EDA graphs such as:

- Loan Approval Status Distribution
- Gender Distribution
- Education Distribution
- Property Area Distribution

**Relationship and Correlation Analysis**

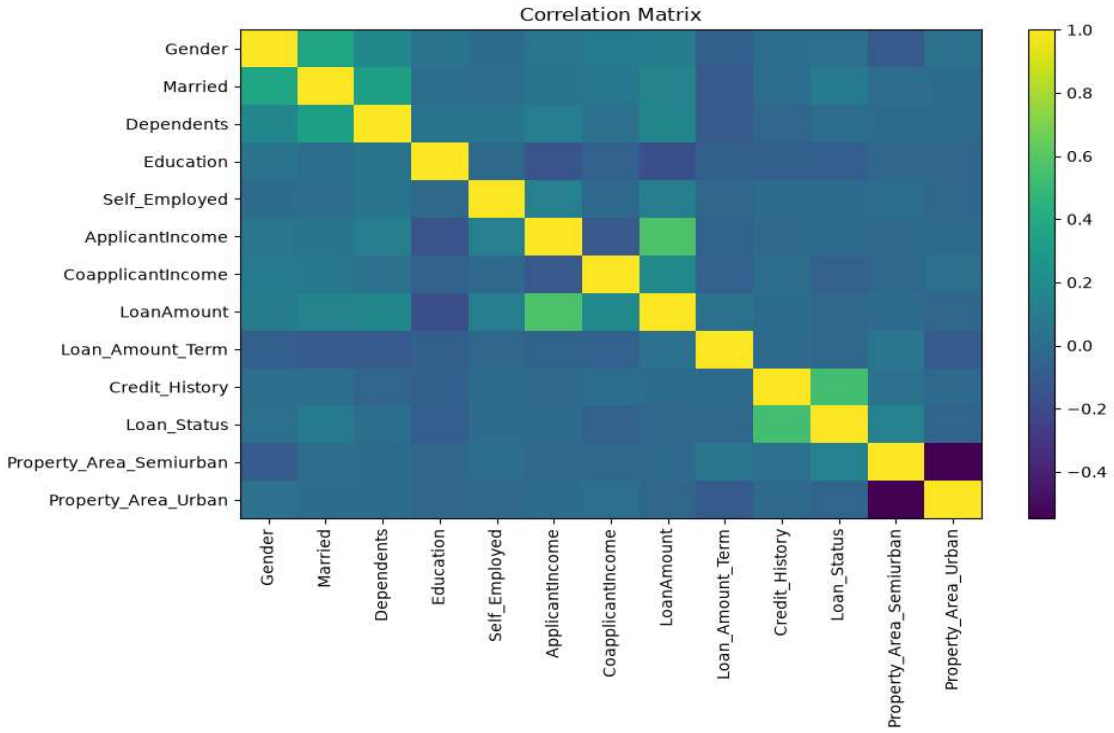
The project also analyzes relationships between applicant attributes and loan approval status using comparative visualizations and a correlation matrix.

**Code:**

```
#Correlation Visualization
plt.figure(figsize=(10, 7))
plt.imshow(correlation, aspect="auto")
plt.colorbar()
plt.xticks(
    range(len(correlation.columns)),
    correlation.columns,
    rotation=90
)
plt.yticks(
    range(len(correlation.columns)),
    correlation.columns
)
plt.title("Correlation Matrix")
plt.tight_layout()
plt.show()
```

**Output:**

***Image 8.5 Depicts*** *Relationship and Correlation Analysis*




Include:

- Credit History vs. Loan Status
- Education vs. Loan Status
- Property Area vs. Loan Status
- Correlation Matrix

## 8.4 Prediction / Analysis Outputs

After preprocessing and feature selection, the dataset was divided into training and testing sets using an 80:20 stratified split. Logistic Regression was then trained and evaluated using different performance metrics.

**Code:**

```
#18. MODEL EVALUATION
#accuracy
accuracy = accuracy_score(y_test, y_pred)
print("Model Accuracy:", accuracy)
print("Model Accuracy (%):", round(accuracy * 100, 2), "%")
```

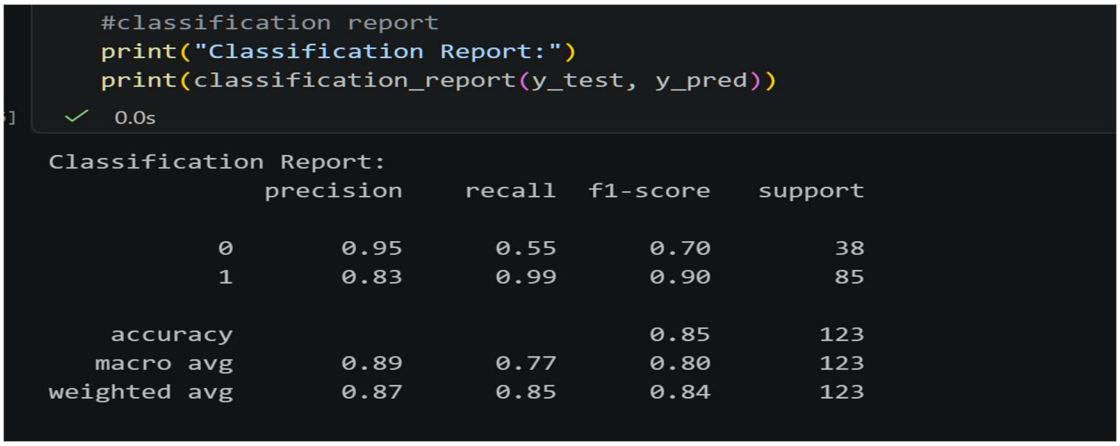
```
#classification report
print("Classification Report:")
print(classification_report(y_test, y_pred))
```

**Output:**

Model Accuracy: 0.8536585365853658

Model Accuracy (%): 85.37 %

***Image 8.6 Depicts*** *Model Evaluation and Loan Prediction Output*




The image contain:

- Train-test split output
- Logistic Regression model
- Accuracy result
- Final prediction output

The obtained model accuracy was **85.37%**, and the sample prediction produced by the system was **Loan Not Approved**.

# CHAPTER 9

# CONCLUSION AND FUTURE ENHANCEMENTS

## 9.1 Conclusion

The **Loan Prediction Data Analysis System** was developed to analyze loan applicant data and identify patterns related to loan approval. The project involved dataset loading, data cleaning, missing-value handling, encoding of categorical attributes, feature scaling, standardization, exploratory data analysis, and visualization.

A Logistic Regression model was implemented using the processed dataset to perform basic loan approval prediction. The model achieved an accuracy of **85.37%** on the test dataset. The results demonstrate that data preprocessing, visualization, and machine learning can be used to analyze loan application data and perform basic prediction.

The project provides a foundation for developing more advanced loan prediction systems using larger datasets, additional features, and improved machine learning techniques.

## 9.2 Limitations

The project has the following limitations:

- The dataset contains **614 records**, which is relatively limited for a real-world loan prediction system.
- Only **Logistic Regression** was implemented for prediction.
- The prediction depends on the features available in the dataset.
- The system is intended for analysis and basic prediction and is not a complete real-world banking loan approval system.
- The model performance may change when applied to different datasets or larger populations.
- Advanced model optimization and deployment were not included in the current implementation.

## 9.3 Future Enhancements

The system can be enhanced in the future by:

- Using larger and more diverse loan datasets.

- Implementing additional machine learning algorithms such as Decision Tree, Random Forest, and XGBoost.
- Comparing multiple models using common evaluation metrics.
- Applying hyperparameter tuning and cross-validation.
- Performing additional feature engineering and outlier analysis.
- Developing an interactive web-based dashboard for loan analysis.
- Creating a user interface where applicant details can be entered to generate predictions.
- Deploying the trained model as a reusable prediction service.

# CHAPTER 10

# REFERENCES

## 10.1 Loan Prediction Dataset

Kaggle. *Loan Prediction Dataset*.
The dataset was used for analyzing loan applicant information and performing loan approval prediction.

**Link: https://www.kaggle.com/**

## 10.2 Pandas Documentation

Pandas Development Team. *Pandas Documentation*.
Used as a reference for data loading, data cleaning, data manipulation, and preprocessing.

**Link: https://pandas.pydata.org/docs/**

## 10.3 NumPy Documentation

NumPy Developers. *NumPy Documentation*.
Used as a reference for numerical operations and data processing.

**Link: https://numpy.org/doc/**

## 10.4 Google Colab

Google. *Google Colaboratory*.
The project implementation and analysis were performed using a Python notebook environment.

**Link: https://colab.research.google.com/**

## 10.5 Reference Book 1

McKinney, Wes. *Python for Data Analysis: Data Wrangling with Pandas, NumPy, and Jupyter*.
3rd Edition, O'Reilly Media, 2022.
ISBN: 978-1098104030.

## 10.6 Reference Book 2

James, Gareth, Daniela Witten, Trevor Hastie, Robert Tibshirani, and Jonathan Taylor. *An Introduction to Statistical Learning: With Applications in Python*.
1st Edition, 2023.
ISBN: 978-3031387463.

## 10.7 GitHub

GitHub. *GitHub Developer Platform*.
Used for storing and managing the project source code and related project files.

**Link: https://github.com/**

# CHAPTER 11

# APPENDIX – SOURCE CODE

## 11.1 Source Code

The complete source code of the **Loan Prediction Data Analysis System** is included in this appendix. The implementation covers dataset loading, data exploration, preprocessing, exploratory data analysis, visualization, model training, evaluation, and prediction.

The source code was implemented using **Python** in a Google Colab/Jupyter Notebook environment.

**Main Implementation Sections**

The source code contains the following major sections:

### 11.1.1 IMPORTING ALL LIBRARIES

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.preprocessing import LabelEncoder, MinMaxScaler, StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix
print("Libraries imported successfully.")
```

### 11.1.2 LOADING THE DATASET

```
df = pd.read_csv("loan.csv")
print("Dataset loaded successfully.")
print("Number of rows and columns:", df.shape)
```

### 11.1.3 DATASET EXPLORATION

```
# Displaying first 5 rows
print("First 5 Rows of the Dataset:")
display(df.head())
```

```
# Displaying Dataset Shape
print("Dataset Shape:", df.shape)
# Displaying dataset information
print("Dataset Information:")
df.info()
# Displaying statistical summary
print("Statistical Summary:")
display(df.describe())
```

### 11.1.4 CHECKING, MISSING VALUE HANDLING

```
# CHECKING MISSING VALUES
print("Missing Values in Each Column:")
print(df.isnull().sum())

#missing values using mode
df["Gender"] = df["Gender"].fillna(df["Gender"].mode()[0])
df["Married"] = df["Married"].fillna(df["Married"].mode()[0])
df["Dependents"] = df["Dependents"].fillna(df["Dependents"].mode()[0])
df["Self_Employed"] = df["Self_Employed"].fillna(df["Self_Employed"].mode()[0])

#missing values using median
df["LoanAmount"] = df["LoanAmount"].fillna(df["LoanAmount"].median())
df["Loan_Amount_Term"]                                                    =
df["Loan_Amount_Term"].fillna(df["Loan_Amount_Term"].median())
df["Credit_History"] = df["Credit_History"].fillna(df["Credit_History"].median())
print("Missing values handled successfully.")

# Verifying that missing values have been handled
print("Missing Values After Cleaning:")
print(df.isnull().sum())
```

### 11.1.5 DUPLICATE & INCONSISTENT DATA HANDLING

```
# Checking duplicate rows
print("Number of Duplicate Rows:", df.duplicated().sum())
# Handling Inconsistent Values
df["Dependents"] = df["Dependents"].replace("3+", 3)
```

### 11.1.6 LABEL ENCODING

```
#lABEL ENCODING
le = LabelEncoder()
model_df["Gender"] = le.fit_transform(model_df["Gender"])
model_df["Married"] = le.fit_transform(model_df["Married"])
model_df["Education"] = le.fit_transform(model_df["Education"])
model_df["Self_Employed"] = le.fit_transform(model_df["Self_Employed"])

# ConvertING Loan Status into numerical form
model_df["Loan_Status"] = le.fit_transform(model_df["Loan_Status"])
print("Label Encoding completed.")
display(model_df.head())
```

### 11.1.7 ONE – HOT ENCODING

```
# 10. ONE-HOT ENCODING
model_df = pd.get_dummies(
    model_df,
    columns=["Property_Area"],
    drop_first=True
)
print("One-Hot Encoding completed.")
print("Columns after One-Hot Encoding:")
print(model_df.columns.tolist())

# Converting Boolean columns into integer values
```

```
for column in model_df.columns:
    if model_df[column].dtype == "bool":
        model_df[column] = model_df[column].astype(int)

print("Boolean values converted to integers.")
```

### 11.1.8 MIN – MAX SCALLING

```
features_to_scale = [
    "ApplicantIncome",
    "CoapplicantIncome",
    "LoanAmount",
    "Loan_Amount_Term"
]
scaler = MinMaxScaler()
model_df[features_to_scale] = scaler.fit_transform(
    model_df[features_to_scale]
)

print("Feature Scaling using Min-Max Scaling completed.")
display(model_df[features_to_scale].head())
```

### 11.1.9 STANDARDIZATION

```
standard_scaler = StandardScaler()
standardized_data = standard_scaler.fit_transform(
    df[
        [
            "ApplicantIncome",
            "CoapplicantIncome",
            "LoanAmount",
            "Loan_Amount_Term"
        ]
    ]
```

```
)
standardized_df = pd.DataFrame(
    standardized_data,
    columns=[
        "ApplicantIncome",
        "CoapplicantIncome",
        "LoanAmount",
        "Loan_Amount_Term"
    ]
)
print("Standardization completed.")
```

### 11.1.10 EXPLORATORY DATA ANALYSIS

```
# 6. EXPLORATORY DATA ANALYSIS
print("Loan Status Distribution:")
print(df["Loan_Status"].value_counts())

print("Gender Distribution:")
print(df["Gender"].value_counts())

print("Education Distribution:")
print(df["Education"].value_counts())

print("Property Area Distribution:")
print(df["Property_Area"].value_counts())
```

### 11.1.11 DATA VISUALIZATION

```
#LOAN APPROVAL STATUS
plt.figure(figsize=(6, 4))
df["Loan_Status"].value_counts().plot(kind="bar")
plt.title("Loan Approval Status")
```

```
plt.xlabel("Loan Status")
plt.ylabel("Number of Applicants")
plt.tight_layout()
plt.show()

#GENDER DISTRIBUTION
plt.figure(figsize=(6, 4))
df["Gender"].value_counts().plot(kind="bar")
plt.title("Gender Distribution")
plt.xlabel("Gender")
plt.ylabel("Number of Applicants")
plt.tight_layout()
plt.show()

#EDUCATION DISTRIBUTION
plt.figure(figsize=(6, 4))
df["Education"].value_counts().plot(kind="bar")
plt.title("Education Distribution")
plt.xlabel("Education")
plt.ylabel("Number of Applicants")
plt.tight_layout()
plt.show()

#PROPERTY AREA DISTRIBUTION
plt.figure(figsize=(6, 4))
df["Property_Area"].value_counts().plot(kind="bar")
plt.title("Property Area Distribution")
plt.xlabel("Property Area")
plt.ylabel("Number of Applicants")
plt.tight_layout()
plt.show()
```

```
#Applicant Income Distribution
plt.figure(figsize=(7, 4))
plt.hist(df["ApplicantIncome"], bins=20)
plt.title("Applicant Income Distribution")
plt.xlabel("Applicant Income")
plt.ylabel("Frequency")
plt.tight_layout()
plt.show()

#Loan Amount Distribution
plt.figure(figsize=(7, 4))
plt.hist(df["LoanAmount"], bins=20)
plt.title("Loan Amount Distribution")
plt.xlabel("Loan Amount")
plt.ylabel("Frequency")
plt.tight_layout()
plt.show()

#Credit history vs Loan status
credit_status = pd.crosstab(
    df["Credit_History"],
    df["Loan_Status"]
)
credit_status.plot(kind="bar", figsize=(7, 4))
plt.title("Credit History vs Loan Status")
plt.xlabel("Credit History")
plt.ylabel("Number of Applicants")
plt.xticks(rotation=0)
plt.tight_layout()
plt.show()
```

```
#Education vs Loan Status
education_status = pd.crosstab(
    df["Education"],
    df["Loan_Status"]
)
education_status.plot(kind="bar", figsize=(7, 4))
plt.title("Education vs Loan Status")
plt.xlabel("Education")
plt.ylabel("Number of Applicants")
plt.xticks(rotation=0)
plt.tight_layout()
plt.show()

#property area vs loan status
property_status = pd.crosstab(
    df["Property_Area"],
    df["Loan_Status"]
)
property_status.plot(kind="bar", figsize=(7, 4))
plt.title("Property Area vs Loan Status")
plt.xlabel("Property Area")
plt.ylabel("Number of Applicants")
plt.xticks(rotation=0)
plt.tight_layout()
plt.show()

#Correlation Analysis
correlation = model_df.corr()
print("Correlation Matrix:")
display(correlation)
```

```
#Correlation Visualization
plt.figure(figsize=(10, 7))
plt.imshow(correlation, aspect="auto")
plt.colorbar()
plt.xticks(
    range(len(correlation.columns)),
    correlation.columns,
    rotation=90
)
plt.yticks(
    range(len(correlation.columns)),
    correlation.columns
)
plt.title("Correlation Matrix")
plt.tight_layout()
plt.show()
```

### 11.1.12 FEATURE SELECTION

```
#14.FEATURE SELECTION
print("Available Model Features:")
print(model_df.columns.tolist())

features = [
    "Gender",
    "Married",
    "Dependents",
    "Education",
    "Self_Employed",
    "ApplicantIncome",
    "CoapplicantIncome",
```

```
    "LoanAmount",
    "Loan_Amount_Term",
    "Credit_History",
    "Property_Area_Semiurban",
    "Property_Area_Urban"
]
X = model_df[features]
y = model_df["Loan_Status"]
print("Selected Features:")
print(X.columns.tolist())
print("\nTarget Variable:")
print("Loan_Status")
```

### 11.1.13 TRAIN – TEST SPLIT

```
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42,
    stratify=y
)
print("X_Train:",X_train.shape)
print("X_Test:",X_test.shape)
print("Y_Train:",y_train.shape)
print("Y_Test:", y_train.shape)
```

### 11.1.14 LOGISTIC REGRESSION MODEL

```
# 16. LOGISTIC REGRESSION MODEL
model = LogisticRegression(max_iter=1000)
model.fit(X_train, y_train)
```

```
print("Logistic Regression model trained successfully.")
```

### 11.1.15 MODEL TRAINING

```
model.fit(X_train, y_train)
```

### 11.1.16 PREDICTION

```
#17. MAKING PREDICTIONS
y_pred = model.predict(X_test)
print("Predictions generated successfully.")
print(y_pred[:10])
```

### 11.1.17 ACCURACY CALCULATION

```
#18. MODEL EVALUATION
#accuracy
accuracy = accuracy_score(y_test, y_pred)

print("Model Accuracy:", accuracy)
print("Model Accuracy (%):", round(accuracy * 100, 2), "%")
```

### 11.1.18 CONFUSION MATRIX

```
#19. CONFUSION MATRIX
cm = confusion_matrix(y_test, y_pred)
print("Confusion Matrix:")
print(cm)

#Confusion matrix visualization
plt.figure(figsize=(6, 5))
plt.imshow(cm)
plt.title("Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("Actual")
```

```
plt.colorbar()
for i in range(len(cm)):
    for j in range(len(cm[i])):
        plt.text(
            j,
            i,
            cm[i][j],
            ha="center",
            va="center"
        )
plt.tight_layout()
plt.show()
```

### 11.1.19 CALSSIFICATION REPORT

```
#classification report
print("Classification Report:")
print(classification_report(y_test, y_pred))
```

### 11.2 Dataset

The project uses the Loan Prediction Dataset in CSV format. The dataset contains 614 records and 13 attributes, with Loan_Status as the target variable.

**Dataset file:** loan.csv